```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd

# 1.1 Load the dataset
df = pd.read_excel('/content/drive/MyDrive/sales_records_dataset.xlsx') # Assuming the f
print(f'Dataset loaded! Shape: {df.shape[0]} rows | {df.shape[1]} columns')

# 1.2 Preview and Info
print(df.head())
print(df.info())

# 1.3 Statistical summary
print(df.describe().round(2))

# 1.4 Check for missing values and duplicates
missing = df.isnull().sum()
print("Missing values per column:\n", missing[missing > 0])
print(f'Number of fully duplicate rows: {df.duplicated().sum()}')
```

```
Dataset loaded! Shape: 1000 rows | 14 columns
  Order_ID     Customer_Name  Gender Region Product_Category Product_Name  \
0  ORD1001      Zainab James  Female   West      Electronics       Laptop
1      NaN     Zainab Okafor    Male   East         Clothing        Jeans
2  ORD1003  Samuel Abdullahi    Male  South         Clothing        Shirt
3  ORD1004    Daniel Balogun  Female   East        Furniture          Bed
4  ORD1005      Ibrahim Khan    Male   West      Electronics       Laptop

   Quantity  Unit_Price  Total_Sales  Discount        Profit  Order_Date  \
0       3.0     21785.0      65355.0    4346.0  11889.607718  2022-07-18
1       2.0     69255.0     138510.0     903.0  17920.557815  2024-09-18
2       9.0    106459.0     958131.0     171.0  56257.963706  2023-09-11
3       5.0    102967.0     514835.0    3286.0  87530.327068         NaN
4       7.0    148887.0    1042209.0     741.0  67011.935599  2022-06-03

  Payment_Method Sales_Rep
0           Cash     Rep_D
1         Online     Rep_A
2            POS     Rep_C
3           Cash     Rep_B
4       Transfer     Rep_D
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Order_ID          955 non-null    object
 1   Customer_Name     956 non-null    object
 2   Gender            949 non-null    object
 3   Region            941 non-null    object
 4   Product_Category  944 non-null    object
 5   Product_Name      961 non-null    object
 6   Quantity          955 non-null    float64
 7   Unit_Price        949 non-null    float64
 8   Total_Sales       942 non-null    float64
 9   Discount          950 non-null    float64
 10  Profit            947 non-null    float64
 11  Order_Date        945 non-null    object
```

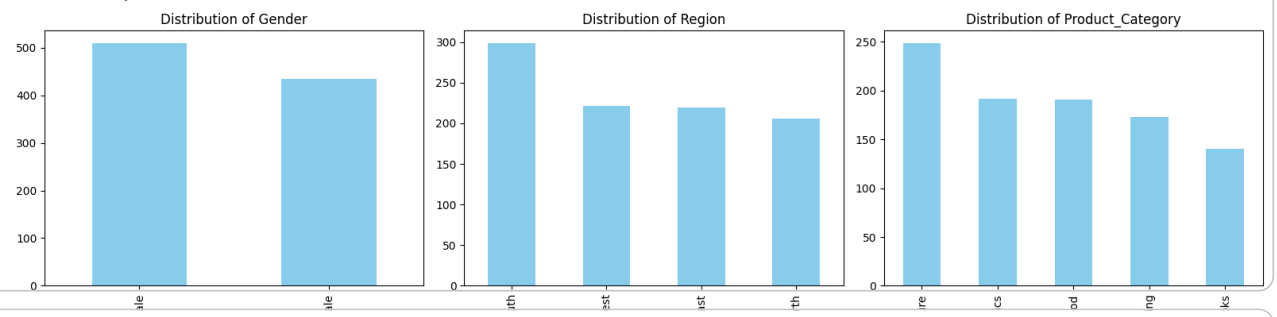
```
 12  Payment_Method    944 non-null    object
 13  Sales_Rep         955 non-null    object
dtypes: float64(5), object(9)
memory usage: 109.5+ KB
None
       Quantity  Unit_Price  Total_Sales  Discount     Profit
count    955.00      949.00       942.00    950.00     947.00
mean       5.53    75752.28    424735.26   2513.22   64645.40
std        2.85    42736.95    339359.57   1463.39   61028.39
min        1.00     2037.00      2118.00      0.00     171.57
25%        3.00    38059.00    133941.50   1271.00   17417.64
50%        6.00    76180.00    357334.50   2497.50   46940.54
75%        8.00   112145.00    621387.25   3796.25   93647.07
max       10.00   149555.00   1492040.00   4996.00  356652.16
Missing values per column:
 Order_ID            45
Customer_Name       44
Gender              51
Region              59
Product_Category    56
```

```
# 2.1 Create a working copy
df_clean = df.copy()

# 2.2 Fix Date column
df_clean['Order_Date'] = pd.to_datetime(df_clean['Order_Date'], errors='coerce')

# 2.3 Handle Missing Values (Numeric)
for col in ['Quantity', 'Unit_Price', 'Total_Sales', 'Profit']:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())
df_clean['Discount'] = df_clean['Discount'].fillna(0)

# 2.4 Handle Missing Values (Categorical)
for col in ['Gender', 'Region', 'Product_Category', 'Payment_Method']:
    df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])

# 2.5 Add derived columns (Feature Engineering)
df_clean['Year'] = df_clean['Order_Date'].dt.year
df_clean['Month'] = df_clean['Order_Date'].dt.month
df_clean['Month_Name'] = df_clean['Order_Date'].dt.strftime('%b')
df_clean['Quarter'] = df_clean['Order_Date'].dt.to_period('Q').astype(str)
df_clean['Profit_Margin'] = ((df_clean['Profit'] / df_clean['Total_Sales']) * 100).round

# Remove rows where date is still missing after coercion
df_clean.dropna(subset=['Order_Date'], inplace=True)
```

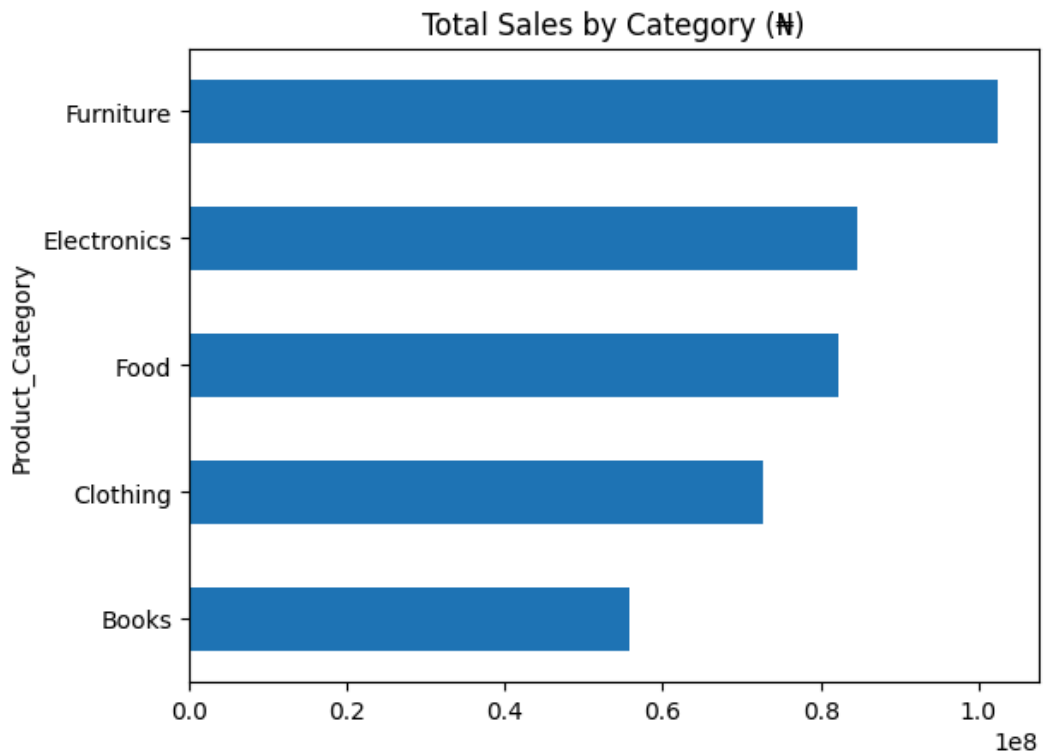
```
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objects as go
import plotly.express as px

# 3.1 Check distribution of key categories
cat_cols = ['Gender', 'Region', 'Product_Category', 'Payment_Method', 'Sales_Rep']
for col in cat_cols:
    counts = df_clean[col].value_counts()
    ratio = counts.max() / counts.min()
    status = 'Balanced' if ratio <= 2.0 else '⚠️ Imbalanced'
    print(f'{col:<20} Ratio: {ratio:.2f} {status}')

# 3.2 Visualization of distributions
fig, axes = plt.subplots(2, 3, figsize=(16, 9))
```

```
axes = axes.flatten()
for i, col in enumerate(cat_cols):
    df_clean[col].value_counts().plot(kind='bar', ax=axes[i], color='skyblue')
    axes[i].set_title(f'Distribution of {col}')
axes[5].set_visible(False)
plt.tight_layout()
plt.show()

# Assign to df_balanced for next stages
df_balanced = df_clean.copy()
```

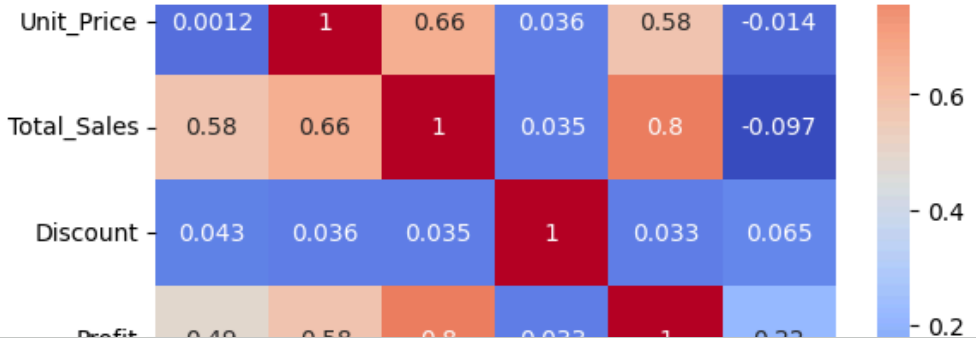
```
Gender               Ratio: 1.17 Balanced
Region               Ratio: 1.45 Balanced
Product_Category     Ratio: 1.78 Balanced
Payment_Method       Ratio: 1.42 Balanced
Sales_Rep            Ratio: 1.11 Balanced
```

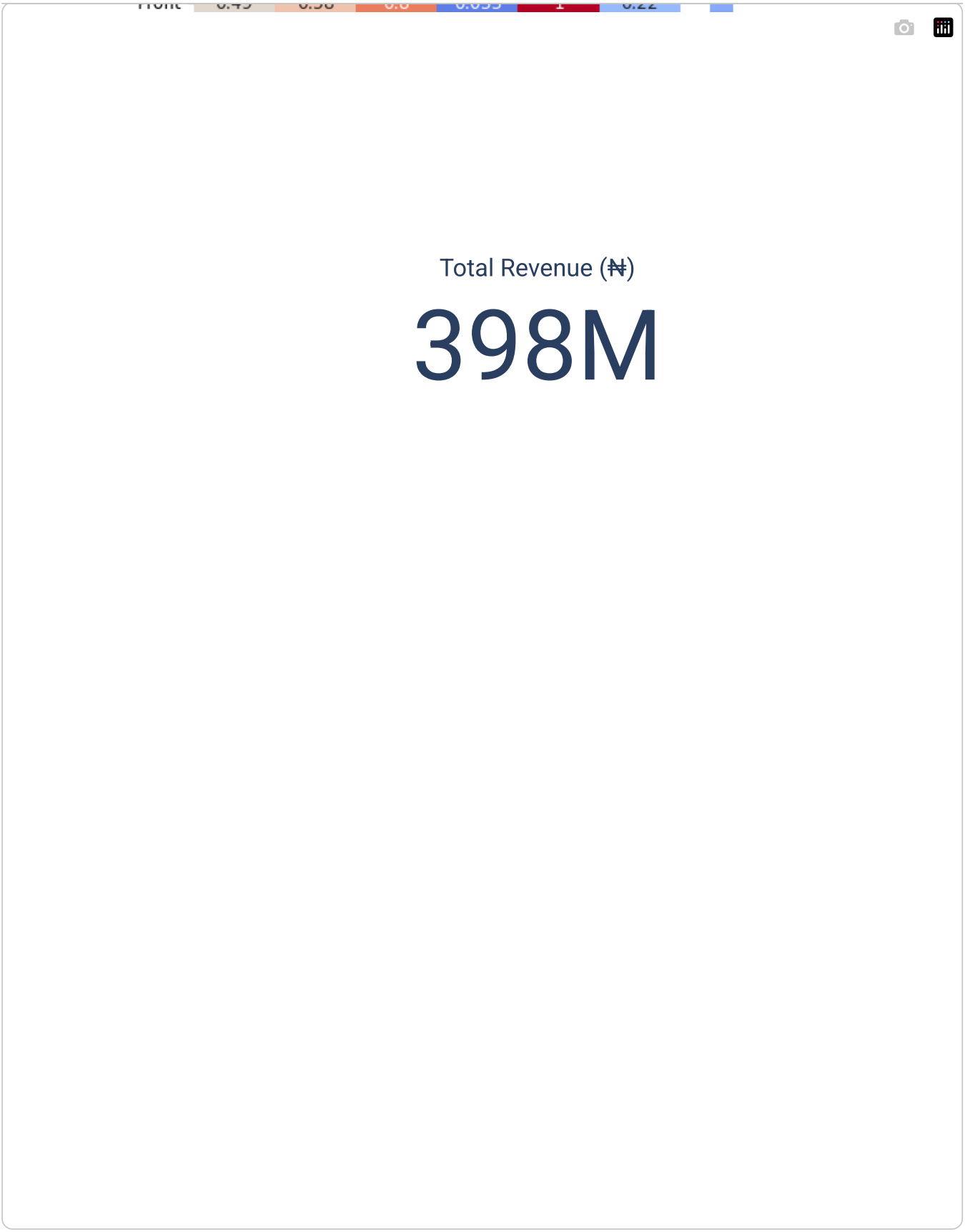



```
# 4.1 Total Sales by Product Category
cat_sales = df_balanced.groupby('Product_Category')['Total_Sales'].sum().sort_values()
cat_sales.plot(kind='barh', title='Total Sales by Category (□)')
plt.show()

# 4.2 Monthly Sales & Profit Trend
monthly = df_balanced.groupby(['Year','Month'])[['Total_Sales','Profit']].sum().reset_i
monthly['Period'] = monthly['Year'].astype(str) + '-' + monthly['Month'].astype(str).st
plt.figure(figsize=(12,5))
plt.plot(monthly['Period'], monthly['Total_Sales'], label='Sales', marker='o')
plt.plot(monthly['Period'], monthly['Profit'], label='Profit', marker='s')
plt.xticks(rotation=45)
plt.legend()
plt.show()

# 4.3 Correlation Heatmap
corr = df_balanced[['Quantity','Unit_Price','Total_Sales','Discount','Profit','Profit_M
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



```
# 5.1 KPI Summary Tiles
total_revenue = df_balanced['Total_Sales'].sum()
total_profit = df_balanced['Profit'].sum()
avg_margin = df_balanced['Profit_Margin'].mean()

fig = go.Figure()
fig.add_trace(go.Indicator(
    mode="number", value=total_revenue,
    title={'text': "Total Revenue (□)"},
    domain={'row': 0, 'column': 0}))
fig.add_trace(go.Indicator(
    mode="number", value=total_profit,
    title={'text': "Total Profit (□)"},
    domain={'row': 0, 'column': 1}))
fig.update_layout(grid={'rows': 1, 'columns': 2})
fig.show()

# 5.2 Category vs Region Heatmap
pivot = df_balanced.pivot_table(values='Total_Sales', index='Product_Category', columns
fig_heat = px.imshow(pivot, text_auto='.1f', title='Sales Heatmap: Category x Region')
fig_heat.show()
```

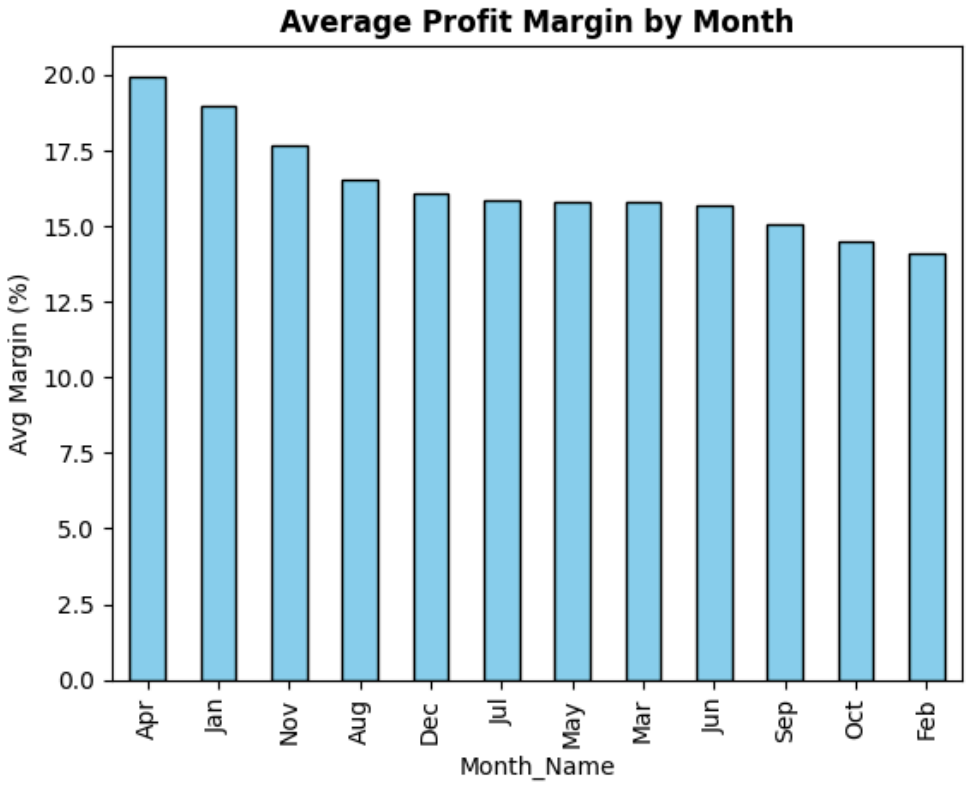





```
# Challenge 1: Average Profit Margin by Month
avg_margin_month = df_balanced.groupby('Month_Name')['Profit_Margin'].mean().sort_value

print("Average Profit Margin per Month:")
print(avg_margin_month.round(2))
```

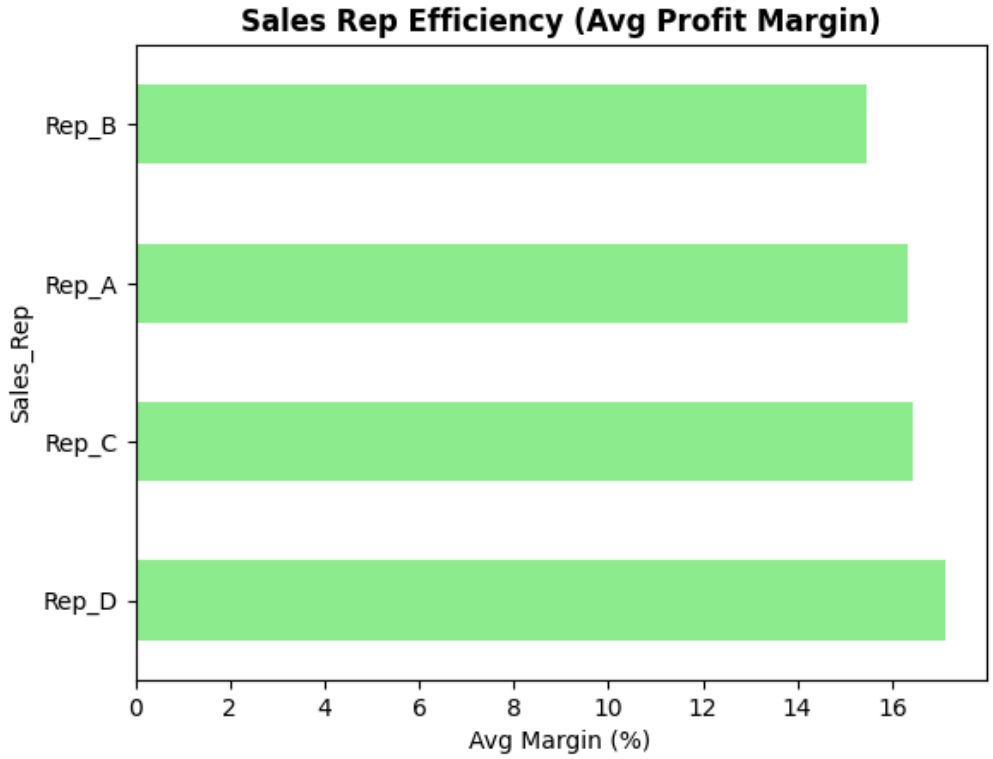
```
# Visualizing the result
avg_margin_month.plot(kind='bar', color='skyblue', edgecolor='black')
plt.title('Average Profit Margin by Month', fontweight='bold')
plt.ylabel('Avg Margin (%)')
plt.show()
```

```
Average Profit Margin per Month:
Month_Name
Apr    19.94
Jan    18.99
Nov    17.64
Aug    16.51
Dec    16.10
Jul    15.84
May    15.78
Mar    15.76
Jun    15.66
Sep    15.08
Oct    14.50
Feb    14.10
Name: Profit_Margin, dtype: float64
```



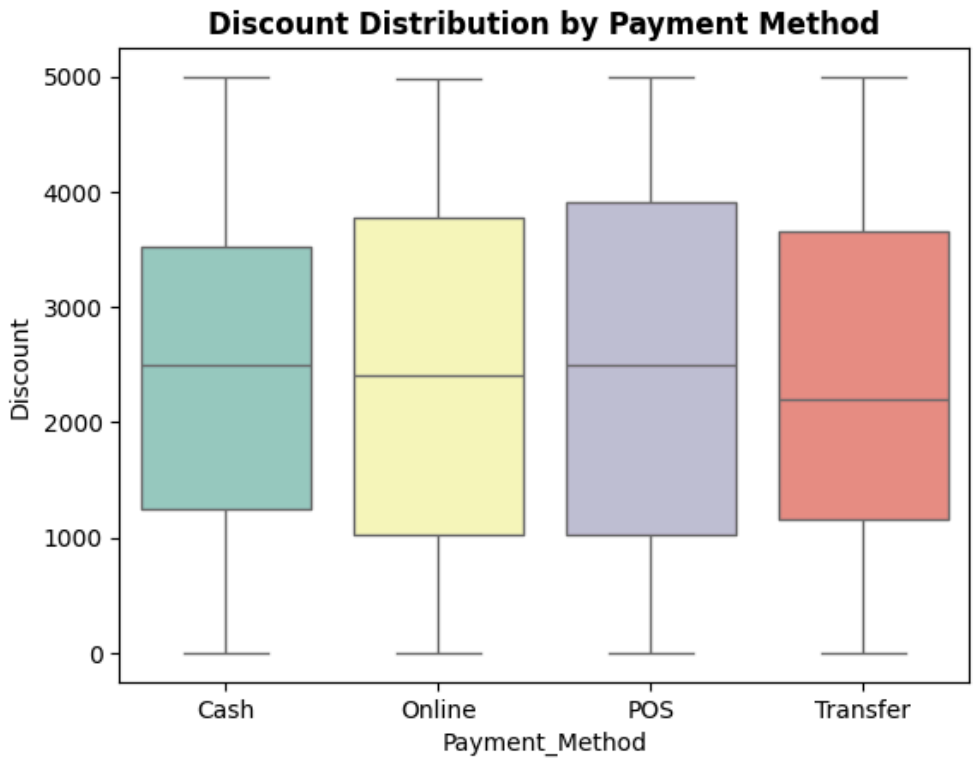

```
# Challenge 2: Best Sales Rep by Profit Margin
rep_margin = df_balanced.groupby('Sales_Rep')['Profit_Margin'].mean().sort_values(ascen

print(f"The Sales Rep with the best average margin is: {rep_margin.index[0]} ({rep_marg

# Comparison Chart
rep_margin.plot(kind='barh', color='lightgreen')
plt.title('Sales Rep Efficiency (Avg Profit Margin)', fontweight='bold')
plt.xlabel('Avg Margin (%)')
plt.show()
```

```
The Sales Rep with the best average margin is: Rep_D (17.12%)
```



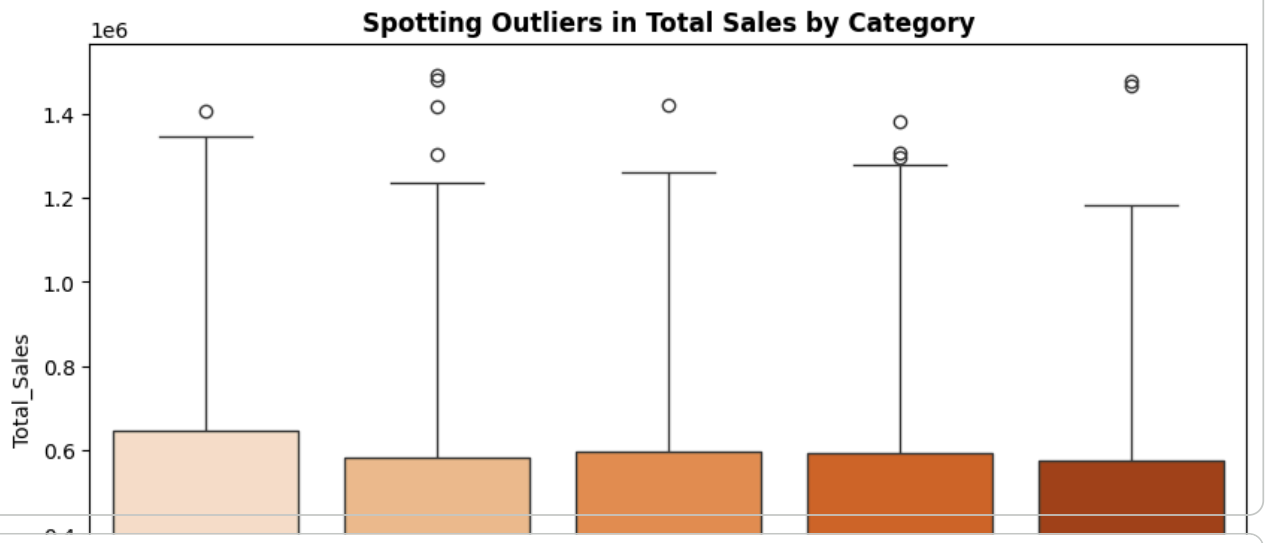

```
# Challenge 3: Discount vs Payment Method
pay_discount = df_balanced.groupby('Payment_Method')['Discount'].describe().round(2)
print("Discount Statistics by Payment Method:")
print(pay_discount)

# Boxplot to see the spread and outliers of discounts
sns.boxplot(data=df_balanced, x='Payment_Method', y='Discount', palette='Set3')
plt.title('Discount Distribution by Payment Method', fontweight='bold')
plt.show()
```

```
Discount Statistics by Payment Method:
                count     mean      std  min      25%     50%      75%     max
Payment_Method
Cash            206.0  2418.38  1445.64  0.0  1244.75  2488.5  3527.75  4996.0
Online          224.0  2381.63  1550.38  0.0  1019.25  2407.5  3770.00  4983.0
POS             293.0  2468.41  1602.03  0.0  1023.00  2491.0  3900.00  4991.0
Transfer        222.0  2338.51  1511.81  0.0  1162.25  2196.0  3657.75  4996.0
/tmp/ipykernel_2795/1175064472.py:7: FutureWarning:


Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0
```




```
# Challenge 4: Identifying Outliers in Total Sales
plt.figure(figsize=(10, 6))
sns.boxplot(data=df_balanced, x='Product_Category', y='Total_Sales', palette='Oranges')
plt.title('Spotting Outliers in Total Sales by Category', fontweight='bold')
plt.show()

print("Note: Look for the dots above the whiskers; the category with the most dots has
```

```
/tmp/ipykernel_2795/1369778439.py:3: FutureWarning:


Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
```




```
# Challenge 5: Quarterly Revenue Growth
quarterly_revenue = df_balanced.groupby(['Year', 'Quarter'])['Total_Sales'].sum().unsta

# Plotting Year-over-Year comparison
quarterly_revenue.plot(kind='line', marker='o', figsize=(10, 5))
```